Assignment 3 — 3D Model Viewer

Advanced Rendering Features

Kamal Nehra 2022MT61979
Computer Graphics

1 Usage

```
./app path/to/model       # load from specified directory
./app                     # load .obj files from current directory
./app -aa 4x4 -shadow -ao 1.0 path/to/model # with AA, shadows, AO
```

The application scans the given directory for `.obj` files and a lights configuration file, then loads them into the scene. Command-line flags `-aa`, `-shadow`, and `-ao` enable anti-aliasing, shadow mapping, and ambient occlusion respectively at startup.

2 Controls

2.1 Trackball Mode (default)

In Trackball mode the cursor is captured and the camera orbits around the object currently under the crosshair.

Input	Action
Mouse drag	Rotate camera around object
Shift + Mouse	Zoom (adjust distance factor)
T	Toggle to View/Edit mode
Tab	Toggle free cursor (unlock mouse from viewport)

2.2 View / Edit Mode

Pressing **T** switches to View/Edit mode. The camera moves freely with WASD and arrow keys. A crosshair ray-cast identifies the object under the cursor.

Input	Action
W / A / S / D	Move camera forward / left / back / right
↑ / ↓	Move camera up / down
Left-click (no drag)	Select / deselect highlighted object
Tab	Toggle free cursor (mouse unlocked from viewport)

2.3 Edit Mode (object selected)

Once an object is selected via left-click, additional manipulation controls become available:

Input	Action
Drag object	Translate object in camera right/up plane
LMB + W/A/S/D / ↑↓	Move camera and object together
Left Ctrl + Mouse	Rotate object around its centre (up/right axes)
Left Ctrl + Shift + Mouse	Rotate about camera front vector
Scroll wheel	Scale object up / down
G	Toggle ground grid
P	Take screenshot

Selection mechanism. Selection uses a click-versus-drag distinction. On mouse-button press the start position is recorded. On release, if the cursor has moved less than a five-pixel threshold, it is treated as a click and the object under the crosshair (identified via the colour-picking FBO) is selected. If the cursor has moved further, it is treated as a drag and selection is not toggled. This replaces the double Caps Lock mechanism from Assignment 2 with a more intuitive single left-click.

Tab – free cursor toggle. Pressing **Tab** toggles between captured and free cursor mode. When the cursor is locked (captured), mouse movement controls the camera and object manipulation. When unlocked, the system cursor is visible and can interact with the ImGui panels without accidentally moving the camera. All mouse callbacks early-out when the cursor is unlocked, preventing unintended viewport input while adjusting UI sliders.

3 User Interface

The application provides three ImGui panels that can be repositioned and docked freely.

3.1 Renderer Panel (Scene Controls)

This panel is the main control hub and displays:

- **Performance**: current FPS and frame time in milliseconds.
- **Camera info**: position, yaw, and pitch values.
- **Trackball mode**: checkbox to toggle between Trackball and View/Edit mode.
- **Show grid**: checkbox (also toggled by G) to display the ground grid.
- **Selection info**: name of the currently hovered and selected object.
- **Ambient strength**: sliders for Blinn-Phong and Cook-Torrance ambient terms (0–1).
- **Anti-Aliasing (MSAA)**: checkbox to enable, with a slider for sample count (up to the hardware maximum). A yellow warning appears when the MSAA framebuffer needs rebuilding.
- **Shadows**: checkbox to enable cascaded shadow maps. When enabled, exposes cascade split slider, cascade overlap, padding, PCF toggle, cascade blend mode, and a cascade debug visualiser.
- **Ambient Occlusion**: checkbox to enable SSAO, with an AO factor slider, debug view toggle, and a "scale entire scene" option.
- **OIT Transparency**: checkbox to enable weighted-blended order-independent transparency.

3.2 Lights Panel

This panel manages directional lights:

- **Add / Delete**: buttons to dynamically add or remove lights at runtime.
- **Enable**: per-light on/off checkbox.

- **Space**: radio buttons to choose Camera-space or World-space. Camera-space lights stay fixed relative to the viewer; world-space lights are transformed by the view matrix each frame.
- **Direction**: three-component slider for the light direction vector.
- **Colour**: colour picker for the light colour.
- **Cast shadow**: per-light checkbox (only available when shadows are globally enabled).

3.3 Material Editor Panel

This panel provides per-model, per-material editing. Models are shown as collapsible tree nodes; within each model, materials are grouped by their material index (shared across meshes that use the same material). For each material:

- **Shading model**: radio buttons to switch between Blinn-Phong and Cook-Torrance.
- **Colour properties**: colour pickers for diffuse (K_d), ambient (K_a), and specular (K_s).
- **Scalar properties**: sliders for shininess (N_s, 0–1000), refractive index (N_i, 1–3), opacity (d, 0–1), and roughness (0.01–1).
- **Texture toggles**: checkboxes to enable or disable diffuse, specular, ambient, normal, and opacity maps (greyed out if the texture was not loaded). A "Flip normal Y" checkbox converts DirectX-convention normal maps to OpenGL convention.
- **Reset**: a button to restore the material to its original loaded values.

Changes to a material are automatically propagated to all sibling meshes that share the same material index.

4 Cascaded Shadow Maps

4.1 Overview

Shadow mapping is implemented using Cascaded Shadow Maps (CSM) with two cascades per shadow-casting directional light. Up to four lights may cast shadows simultaneously. Each cascade renders a 2048×2048 depth map.

4.2 Cascade Partitioning

The camera's view frustum is split into two depth ranges controlled by a user-adjustable split distance. Cascade 0 covers the near plane to the split distance; Cascade 1 covers the split distance to the far plane. An overlap parameter allows the two ranges to extend slightly into each other, reducing hard transition artefacts at the cascade boundary.

4.3 Light Matrix Construction

For each cascade, the camera-space frustum corners for that depth range are computed and transformed into light space. The light's view matrix is constructed by looking along the light direction from above the scene. An axis-aligned bounding box of the frustum corners in light space determines the orthographic projection extents. A padding factor expands the bounding box slightly to avoid edge clipping. The near plane of the orthographic projection is extended backwards to include any scene geometry that might cast shadows into the frustum but lies outside it, by iterating over all visible models and accounting for their bounding-sphere radii.

4.4 Rendering

Shadow map rendering uses front-face culling (`glCullFace(GL_FRONT)`) to reduce shadow acne. Each cascade's depth texture is rendered by attaching it to a shared FBO and drawing all visible

scene geometry using a minimal vertex-only shadow shader. The resulting depth textures use hardware shadow comparison (`GL_COMPARE_REF_TO_TEXTURE`) for efficient sampling.

4.5 Shadow Lookup in the Fragment Shader

In the main fragment shader, each fragment determines its cascade by comparing its view-space depth against the split thresholds. The fragment is then projected into the chosen cascade's light clip space. A slope-based bias is computed from the dot product of the surface normal and the light direction to reduce both shadow acne and peter-panning. When a fragment's projection falls outside the valid region of cascade 0 (checked by boundary coordinates), the shader falls back to cascade 1, and an optional blend mode can interpolate between the two cascades in the overlap zone.

4.6 PCF Filtering

When PCF is enabled, each shadow lookup samples a 3×3 neighbourhood of the shadow map using `sampler2DShadow`, averaging nine hardware-compared samples to produce soft shadow edges. When disabled, a single sample is used for hard shadows.

4.7 Debug Visualisation

A cascade debug mode tints fragments with distinct colours: red for cascade 0, blue for cascade 1, and green where both cascades overlap, making it easy to verify the split distance and overlap settings.

5 Anti-Aliasing

Anti-aliasing is implemented via Multisample Anti-Aliasing (MSAA) using an off-screen multisample framebuffer. When enabled, the application creates a multisample colour texture (`GL_TEXTURE_2D_MULTISAMPLE`) and a multisample depth-stencil renderbuffer, both allocated at the current framebuffer resolution with the user-specified sample count. All scene rendering is directed to this MSAA FBO instead of the default framebuffer. After the scene pass completes, the multisample FBO is resolved to the default framebuffer using `glBlitFramebuffer`. The sample count is adjustable at runtime through the UI slider; changing it sets a rebuild flag, and the FBO is reconstructed on the next frame. The maximum sample count is queried from the driver via `GL_MAX_SAMPLES`.

6 Screen-Space Ambient Occlusion (SSAO)

6.1 Overview

Ambient occlusion darkens regions where geometry is closely packed or occluded, adding depth and contact shadows without additional light sources. The implementation follows a three-pass screen-space approach.

6.2 Geometry Pass

A dedicated G-buffer FBO is rendered with two colour attachments: view-space position (`GL_RGBA16F`) and view-space normal (`GL_RGBA16F`), plus a depth renderbuffer. All visible scene geometry is drawn using a geometry shader that outputs view-space position and normal per fragment.

6.3 SSAO Compute Pass

A fullscreen quad is drawn, sampling the G-buffer. For each fragment, 64 hemisphere sample points (pre-generated at initialisation using a cosine-weighted distribution biased toward the surface) are oriented around the surface normal using a TBN matrix constructed from a tiled 4×4 random rotation noise texture. Each sample is offset from the fragment's view-space position, projected back into screen space using the projection matrix, and the stored depth at that screen coordinate is compared against the sample's depth. A range-check falloff prevents distant geometry from contributing false occlusion. The occlusion factor is accumulated and normalised, yielding a per-pixel AO value.

6.4 Blur Pass

The raw AO buffer is blurred using a simple box filter to remove the noise pattern introduced by the 4×4 tiling. The blurred output is stored in a separate single-channel texture.

6.5 Application in Shading

The blurred AO texture is sampled in the main fragment shader using the fragment's screen-space UV coordinates derived from its clip-space position. The occlusion value is multiplied by a user-controlled AO factor. Two application modes are provided: in the default mode, only the ambient term is attenuated by the AO factor; in "scale entire scene" mode, both the ambient and direct lighting terms are scaled, producing a more dramatic effect. A debug mode renders the AO buffer directly as greyscale for visual inspection.

7 Order-Independent Transparency (OIT)

7.1 Overview

Transparent surfaces (meshes with opacity $d < 1$) require special handling because standard alpha blending is order-dependent. The application implements Weighted Blended Order-Independent Transparency, which approximates correct transparency compositing in a single geometry pass without requiring depth sorting.

7.2 Rendering Pipeline

The rendering pipeline first draws all opaque geometry (meshes with $d \geq$ a configurable threshold) into either the MSAA FBO or the default framebuffer, writing colour, depth, and stencil as usual. Then, transparent geometry is drawn in a separate OIT pass:

1. **OIT begin**: The opaque depth buffer is blitted into the OIT FBO. Two render targets are set up — an accumulation texture (`GL_RGBA16F`) cleared to zero, and a revealage texture (`GL_R8`) cleared to one. Depth writes are disabled and additive blending is configured: the accumulation attachment uses `GL_ONE, GL_ONE` blending, and the revealage attachment uses `GL_ZERO, GL_ONE_MINUS_SRC_COLOR`.
2. **Fragment shader**: Each transparent fragment computes its lit colour and alpha normally, then produces a weight w based on colour intensity, alpha, and a depth-dependent factor $0.03/(10^{-5} + (z/200)^4)$. The accumulation output is $(C \cdot \alpha) \cdot w$ with alpha component $\alpha \cdot w$; the revealage output is α.
3. **OIT composite**: A fullscreen pass reads both textures. Fragments where the revealage is approximately 1.0 (fully transparent) are discarded. Otherwise, the average colour is reconstructed as accum.rgb/accum.a, and blended with the background using alpha $= 1 -$ revealage.

When OIT is disabled, transparent meshes fall back to conventional sorted alpha blending with depth writes disabled.

7.3 Fallback

If OIT is toggled off via the UI, transparent geometry is rendered using standard alpha blending (`GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA`) with depth writes disabled, which may produce ordering artefacts but avoids the extra FBO overhead.

8 Additional Features from Assignment 2

The following features from Assignment 2 remain unchanged and are still present:

- Model loading via Assimp with per-mesh GPU state (VAO/VBO) and per-model transforms.
- Blinn-Phong and Cook-Torrance shading with texture maps (diffuse, specular, ambient, normal, opacity).
- Camera/world-space directional lights loaded from a config file.
- Colour-based object picking via an off-screen FBO.
- Stencil-based selection outline rendering.
- Bounding-sphere ray intersection for hover detection.
- Screenshot capture (P key).